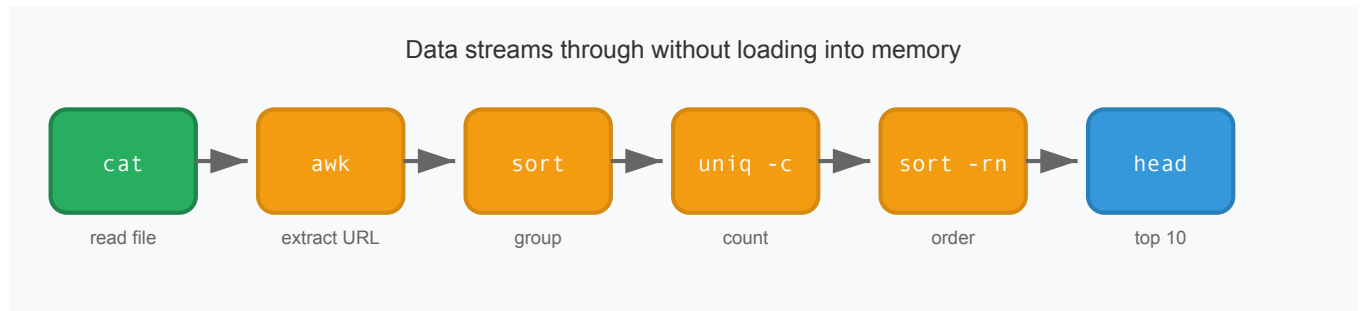


Unix Pipelines

 systemdesignschool.io/fundamentals/unix-pipelines



The previous article introduced batch and stream processing as ways to create derived data. Before we dive into distributed systems like MapReduce and Spark, let's understand the conceptual foundation they built upon: Unix pipelines.

In the 1970s, Unix engineers solved data processing with a simple idea: chain small programs together. Each program reads from standard input (stdin), transforms the data, and writes to standard output (stdout). The shell connects them with pipes (`|`). This design shaped how we think about data processing at scale.

A Concrete Example: Analyzing Web Logs

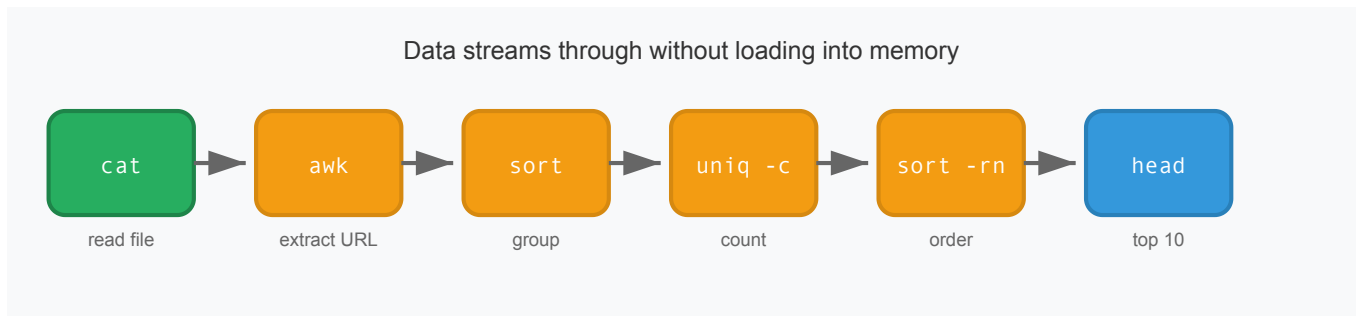
Suppose you manage a web server and want to find the 10 most frequently accessed URLs from yesterday's access log. The log file has millions of lines, each with a timestamp, client IP, HTTP method, URL path, and response code.

Here's a Unix pipeline that answers the question:

```
cat access.log | awk '{print $7}' | sort | uniq -c | sort -rn | head -10
```

Let's walk through what happens at each step:

1. **cat access.log** — reads the log file and streams lines to stdout
2. **awk '{print \$7}'** — extracts just the URL field (the 7th column) from each line
3. **sort** — alphabetically sorts all URLs, grouping identical ones together
4. **uniq -c** — counts consecutive duplicate lines (since identical URLs are now adjacent)
5. **sort -rn** — sorts numerically by count, highest first
6. **head -10** — takes only the top 10 results



Notice something important: each program does one thing. **sort** doesn't know about URLs—it just sorts lines. **uniq** doesn't know about logs—it just counts duplicates. Yet the combination analyzes web traffic patterns.

Why This Model Scales

Notice that the entire log file never loads into memory at once. Data flows through the pipeline in chunks. Each step processes lines as they arrive, then passes results downstream.

Consider a 100GB log file. A naive approach would load it into memory, build a hash map of URL counts, sort by count, and return the top 10. That requires 100GB+ of RAM. The pipeline approach streams through the file using minimal memory—though the **sort** step needs to buffer data (typically spilling to disk for large inputs).

This principle generalizes: **streaming through data beats loading into memory** when datasets exceed available RAM.

Properties That Make Pipelines Powerful

Composability. Simple tools combine to solve complex problems. Need the top URLs by response time instead of frequency? Swap **awk '{print \$7}'** for one that extracts both URL and response time, add an aggregation step. The other pieces stay the same.

Process isolation. Each program runs in its own process with its own memory space. One step can be CPU-bound (like **sort**) while another waits on I/O (like **cat** reading from disk). The OS kernel schedules them independently, achieving parallelism on a single machine.

Fault tolerance. If the **sort** step crashes, you can rerun just that step—its input (the output of **awk**) can be regenerated from the original file. You don't need to start over from the beginning. This restartability property becomes crucial in distributed systems where machine failures are common.

Implicit backpressure. When one step produces data faster than the next step can consume it, the pipe buffer fills up. The fast producer blocks until the slow consumer catches up. This prevents memory exhaustion without explicit coordination.

The Single-Machine Limitation

Unix pipelines hit a wall: they run on one machine. That 100GB log processes fine, but slowly—sorting might take hours. A 10TB dataset won't fit on one disk. A petabyte is out of the question.

What if we could take these same ideas—streaming data through transformations, process isolation, restartability—and distribute them across hundreds of machines?

That's exactly what MapReduce does. From here, Google adapts Unix pipeline thinking to clusters of commodity servers, enabling analysis of datasets too large for any single machine.